

DirectNVM: Hardware-accelerated NVMe SSDs for High-performance Embedded Computing

YU ZOU, University of Central Florida

AMRO AWAD, North Carolina State University

MINGJIE LIN, University of Central Florida

With data-intensive artificial intelligence (AI) and machine learning (ML) applications rapidly surging, modern high-performance embedded systems, with heterogeneous computing resources, critically demand low-latency and high-bandwidth data communication. As such, the newly emerging NVMe (Non-Volatile Memory Express) protocol, with parallel queuing, access prioritization, and optimized I/O arbitration, starts to be widely adopted as a de facto fast I/O communication interface. However, effectively leveraging the potential of modern NVMe storage proves to be nontrivial and demands fine-grained control, high processing concurrency, and application-specific optimization. Fortunately, modern FPGA devices, capable of efficient parallel processing and application-specific programmability, readily meet the underlying physical layer requirements of the NVMe protocol, therefore providing unprecedented opportunities to implementing a rich-featured NVMe middleware to benefit modern high-performance embedded computing.

In this article, we present how to rethink existing accessing mechanisms of NVMe storage and devise innovative hardware-assisted solutions to accelerating NVMe data access performance for the high-performance embedded computing system. Our key idea is to exploit the massively parallel I/O queuing capability, provided by the NVMe storage system, through leveraging FPGAs' reconfigurability and native hardware computing power to operate transparently to the main processor. Specifically, our DirectNVM system aims at providing effective hardware constructs for facilitating high-performance and scalable userspace storage applications through (1) hardening all the essential NVMe driver functionalities, therefore avoiding expensive OS syscalls and enabling zero-copy data access from the application, (2) relying on hardware for the I/O communication control instead of relying on OS-level interrupts that can significantly reduce both total I/O latency and its variance, and (3) exposing cutting-edge and application-specific weighted-round-robin I/O traffic scheduling to the userspace.

To validate our design methodology, we developed a complete DirectNVM system utilizing the Xilinx Zynq MPSoC architecture that incorporates a high-performance application processor (APU) equipped with DDR4 system memory and a hardened configurable PCIe Gen3 block in its programmable logic part. We then measured the storage bandwidth and I/O latency of both our DirectNVM system and a conventional OS-based system when executing the standard FIO benchmark suite [2]. Specifically, compared against the PetaLinux built-in kernel driver code running on a Zynq MPSoC, our DirectNVM has shown to achieve up to 18.4× higher throughput and up to 4.5× lower latency. To ensure the fairness of our performance comparison, we also measured our DirectNVM system against the Intel SPDK [26], a highly optimized userspace asynchronous NVMe I/O framework running on a X86 PC system. Our experiment results have

The views, opinions and/or findings expressed are those of the author and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government. This research was developed with funding from the Defense Advanced Research Projects Agency (DARPA).

Authors' addresses: Y. Zou and M. Lin, University of Central Florida, Orlando, FL, US; emails: yuzou@knights.ucf.edu, mingjie.lin@ucf.edu; A. Awad, North Carolina State University, Raleigh, NC, US; email: ajawad@ncsu.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

1539-9087/2022/02-ART9 \$15.00

<https://doi.org/10.1145/3463911>

shown that our DirectNVM, even running on a considerably less powerful embedded ARM processor than a full-scale AMD processor, achieved up to 2.2× higher throughput and 1.3× lower latency. Furthermore, by experimenting with a multi-threading test case, we have demonstrated that our DirectNVM's weighted-round-robin scheduling can significantly optimize the bandwidth allocation between latency-constraint frontend applications and other backend applications in real-time systems. Finally, we have developed a theoretical framework of performance modeling with classic queuing theory that can quantitatively define the relationship between a system's I/O performance and its I/O implementation.

CCS Concepts: • **Computer systems organization** → *System on a chip*; • **Hardware** → **External storage**; *Memory and dense storage*; *Emerging interfaces*; **Hardware accelerators**; **Reconfigurable logic applications**;

Additional Key Words and Phrases: NVMe, SSDs, FPGA, high-Throughput high-Performance computing

ACM Reference format:

Yu Zou, Amro Awad, and Mingjie Lin. 2022. DirectNVM: Hardware-accelerated NVMe SSDs for High-performance Embedded Computing. *ACM Trans. Embedd. Comput. Syst.* 21, 1, Article 9 (February 2022), 24 pages.

<https://doi.org/10.1145/3463911>

1 INTRODUCTION

While the computing concurrency and the peak performance of a modern embedded computing system have surged ahead in recent years, constrained storage I/O and network bandwidth have often impeded its responsiveness and throughput [17]. This is especially problematic for increasingly more important IoT and edge computing applications—including computer vision and deep learning workloads—that interact continuously with large data storage systems while being subjected to stringent performance demands. As such, the pivot of modern embedded computing has been quickly shifting from computing logic to data storage and networking recently [8]. If hardware cost was no consideration, to mitigate such I/O performance bottlenecks and maximize the computing performance of many embedded computing applications, then one conceptually straightforward remedy is to utilize large quantities of main memory jointed with the fastest multi-core processors or adopt novel computing methodologies such as in-memory computing.

However, given the limited quantity of main memory and the strict energy budget for modern embedded computing, to effectively circumvent the above data path bottlenecks, possibly the only viable near-term solution is to augment processing systems with near-memory-speed secondary storage. As such, NAND-flash-based **solid-state drives (SSDs)** are being widely adopted in high-performance embedded computing systems to replace **hard disk drives (HDDs)** to improve the performance and energy efficiency of storage devices.

Not surprisingly, much research focus has been placed on how to more effectively exploit the SSD storage parallelism [12, 18, 21]. **Non-Volatile Memory Express (NVMe)** technology, with a novel data transport abstraction layer, has been designed from scratch for non-volatile devices, such as SSDs and Persistent Memory [10], which are directly attached to many-core CPUs or GPUs to sustain ultra-high data communication bandwidth. In particular, leveraging the standard high-speed PCIe serial bus, the NVMe-based SSD storage is capable of providing **gigabyte-per-second (GBps)** linear I/O throughput, as well as achieving tens of microseconds latency for random 4K data accesses, which is significantly faster than the standard SATA interface. Technically, there are at least three key advantages of adopting the NVMe-based storage systems in high-performance embedded computing.

- First, the NVMe protocol possesses multiple unique features such as multiple queuing, IO combining, process prioritization, and virtualized multi-path I/Os. As such, the NVMe can often achieve an increase of about four orders of magnitude in queuing capability when compared to legacy SAS and SATA protocols.
- Second, the NVMe is a highly scalable **NUMA-optimized (Non-Uniform Memory Access)** storage protocol, and therefore can effectively interconnect many-core processing engines with the high-bandwidth storage subsystem. This is especially important for today's embedded systems with heterogeneous computing resources because it allows multiple CPU cores and various computing accelerators to share the ownership of queues, their priority, as well as arbitration mechanisms and atomicity of the commands.
- Third, NVMe SSDs can scatter/gather commands and process them out of turn to offer far higher IOPS and significantly lower data latencies. All these can even potentially enable or lead to more advanced storage concepts such as **NVMe over Fabrics (NVMe-oF)**, a protocol specification designed to connect hosts to storage across a network fabric using the NVMe protocol.

All these aforementioned NVMe-related performance benefits, if fully realized in embedded computing, can not only accelerate important data-intensive applications, but also enable new functionalities and capabilities for real-time workload processing in the data center, as well as in the edge computing. Unfortunately, with the explosion of “big” data, without major innovations, directly adopting NVMe-based storages into conventional computing systems will still suffer from lower performance, higher latencies, and poor QoS due to their *legacy storage protocols* as well as *inflexible user controls*.

In parallel to the deployment of NVMe, modern FPGA devices are getting more widely adopted in embedded applications, such as edge computing, autonomous driving, augmented reality, remote monitoring, and recognition, and even high-performance computing applications, such as graph analysis and databases. This trend is largely due to FPGA's fine-grained hardware parallelism, flexible reconfigurability, and ultra-high energy efficiency. Traditionally, FPGAs and processors are connected via PCIe, and many improvements are made to improve memory address space sharing and cache coherency, such as IBM CAPI SNAP [6] and Intel OPAAE [11]. With FPGAs being widely used, there is a trend of heterogeneous SoCs packaging an FPGA and a processor within the same die, such as Xilinx Zynq family, Intel Stratix SoC family, Intel HARP. By coupling an FPGA with a processor, more accelerating opportunities are exposed to FPGAs, such that not only data processing tasks but also some OS functionalities can be offloaded to an FPGA. In doing so, an NVMe-based SSD storage system can effectively integrate FPGA-based accelerators with its flash storage, therefore improving both its energy efficiency and performance. Probably even more excitingly, all these new FPGA-augmented features can further enable new functionalities and capabilities, such as reliability-related and security-related applications, for real-time workload processing in the data center and at the edge of Internet.

In this work, we designed a novel architecture, *DirectNVM*, that tightly incorporates a userspace I/O framework with a thin kernel stack bypassing block I/O layer, in which almost all functionalities of a standard NVMe driver are offloaded to the FPGA hardware to significantly reduce software overhead. To put our work into perspective, we compared our DirectNVM and other existing NVMe systems in Table 1 with I/O throughput, latency, and multi-threading support, and so on. Furthermore, in Figure 1, we depicted the overall architecture comparison between a baseline system and our DirectNVM system. Here, the most striking difference between our DirectNVM and a conventional NVMe storage kernel stack lies in the data path a typical I/O request will traverse inside both systems. While in a conventional NVMe system, an I/O request will traverse a complete

Table 1. Comparison of DirectNVM and Other Existing Systems

	Kernel Driver	SPDK [26]	FastPath [22]	DirectNVM
Throughput	Low	High	High	High
Latency	High	Low	Low	Low
Priority-based Scheduling	RR ¹	RR ¹	RR ¹	WRR²
NVMe Driver	OS Kernel Space	OS User Space	Hardware	Hardware
Support for Multi-Thread	Yes	Yes	No	Yes

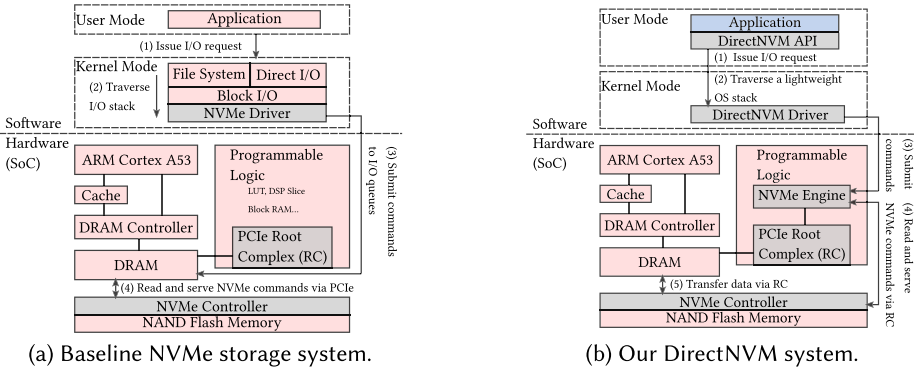
¹RR: round robin.²WRR: weighted round robin.

Fig. 1. Overall architecture comparison between the baseline and our DirectNVM system on ARM-FPGA SoCs.

OS stack before finally reaching the PCIe root complex in a SoC platform; in our DirectNVM system, the same I/O request will pass by a very thin DirectNVM kernel driver, immediately engage a rich-featured hardware NVMe engine, and finally reach the same PCIe root complex. Specifically, our DirectNVM work attempts to achieve two objectives:

- Our first key objective is to investigate how to efficiently combine and delegate both acceleration and OS functionality onto the FPGA fabric while bypassing the multi-core processing system to maximize the storage performance.
- Our second key objective is to realize the application-specific and fine-grained QoS in NVMe SSD storage. Conventionally, NVMe scheduling logic schedules multiple request queues in a **round-robin (RR)** way with no control of QoS such that multiple running applications compete for NVMe bandwidth. Consequently, applications with different storage access patterns unfairly suffer different slow-downs. Very importantly, in a mission-critical or real-time system, a thread/application may require reserving a certain amount of bandwidth to meet its specific latency constraint, which is not compatible with the current standard round-robin priority scheduling offered by all the mainstream NVMe I/O frameworks. To this end, our DirectNVM architecture, leveraging the abundant hardware parallelism of modern FPGAs, allows a user application to transparently extend fine-grained and application-specific I/O control and to utilize new-generation NVMe SSDs' built-in **weighted-round-robin (WRR)** scheduling, therefore achieving higher overall data throughput.

In this work, we claim the following contributions:

- Designed and open-sourced an efficient FPGA NVMe controller IP directly connecting FPGAs with NVMe SSDs, compliant with NVMe 1.3 protocol. These IPs are specifically designed to be self-managed and not rely on processors, such that they can be adopted on any platforms with or without embedded processors, as well as can be parallelized in any degree given sufficient hardware resource.
- Proposed an FPGA-accelerated userspace I/O framework, DirectNVM, that provides flexible APIs and a lightweight software stack to improve NVMe I/O latency and throughput. Leveraging a self-managed controller, DirectNVM is quite friendly to multi-core architectures because it can effectively avoid multiple cores' simultaneous NVMe completion polling via shared DRAM ports.
- Proposed a differentiated I/O service by exposing the weighted-round-robin priority control to each I/O traffic instead of each core to provide more flexible QoS control.
- Prototyped DirectNVM on a Xilinx SoC attached with a commercial NVMe SSD. The datapath latency is shortened by up to 4.5 $\times$, and the system throughput is improved by up to 12.6 $\times$, when measured against a standard NVMe implementation provided commercially. Compared with state-of-the-art Intel SPDK, DirectNVM achieves up to 2.2 $\times$ higher throughput and 1.3 $\times$ shorter latency.

The rest of the article is divided as follows: Section 2 provides a brief overview of the standard NVMe system architecture, which serves as our test baseline. Section 3 presents the architecture of our proposed DirectNVM system. Specifically, we motivate this study and highlight various innovations in framework design and architectural implementation of our DirectNVM. In Section 4, we present more details about both the I/O framework and key hardware modules, on the design and implementation of our DirectNVM architecture. Section 5 describes our experimental setup and evaluation methodology. Section 6 presents a mathematical model to the DirectNVM's queuing performance, which guided us to effectively determine the optimal queue size given a fixed limit of hardware usage, and a detailed results analysis.

2 BASELINE NVME SYSTEM ARCHITECTURE

Standard NVMe architecture, consisting of multiple abstraction layers, is designed to efficiently access the storage built with non-volatile devices, ranging from the NAND flash technology to other emerging persistent memory technologies. Like other storage media, SSDs are managed as block devices in the OS and I/O requests are performed through multiple abstraction layers. A file system allows a user application to access the SSD without directly interacting with the physical sectors and **logical block addresses (LBAs)**. OS system calls (e.g., `ioctl`, `read`, `write`) are used to interpret user-level requests as system functions at the kernel level and perform I/Os on behalf of the requesting applications. In essence, the NVMe protocol stack is responsible for communicating and manipulating I/O requests from the applications to the storage devices while providing services such as error handling, profiling, and fair scheduling to improve I/O performance.

In the rest of this section, we briefly but systematically describe some key features of the standard NVMe-based storage system in three aspects: (1) the NVMe data and control paths, (2) the NVMe device driver and hardware controller, and (3) the standard I/O queuing mechanism of NVMe, all depicted in Figure 1(a).

2.1 NVMe Data and Control Paths

In a standard NVMe-based storage system, as depicted in Figure 1(a), on the software side, user applications, through file system or direct I/O, first submit all I/O requests to the **block I/O layer**

(BIO) (Step 1), traverse the whole I/O stack, and eventually reach the NVMe driver running in kernel mode (Step 2). The NVMe device driver retrieves those I/O requests, creates corresponding NVMe commands, and submits them to the controller to be served (Step 3). Finally, the NVMe controller, through DRAM memory and the **Root Complex (RC)** IP of the FPGA, reads the commands from predefined memory addresses and serves the requests (Step 4). All these predefined memory locations are allocated during initialization, where the NVMe driver creates in-memory queues from which the controller can fetch commands. In addition to a required pair of admin queues, the driver normally creates a pair of I/O queues for each processing core to facilitate multi-core systems. Each I/O queue pair consists of one submission and one completion queue, where submission and completion commands are enqueued, respectively. Overall, the stack is designed to achieve I/O scalability such that all the cores can process I/O requests in parallel.

2.2 NVMe Device Driver and Hardware Controller

NVMe-based data transfer can be approximately abstracted as an interaction between an NVMe device driver on the software side and a hardware controller on the hardware side. The device driver synthesizes NVMe commands for incoming I/O requests and the NVMe controller, residing inside the SSD, is responsible for fetching and decoding I/O commands, scheduling low-level accesses to the flash drive, and other operations such as garbage collection. For each I/O command, the NVMe driver allocates a source/destination data buffer, termed as **physical region pages (PRP)**, in the system memory to write data to or read data from. Besides, each command contains a starting **logical block address (LBA)**, the transfer size, and an opcode. At the same time, the NVMe controller (1) configures all queue structures per admin commands, (2) executes received I/O commands, (3) translates a logical block address into a physical block address, (4) conducts error detection of all memory cells in the associated storage device, and (5) responds to the device driver for the successful or unsuccessful completion of requested I/O operations.

We now summarize the key steps contained in one complete NVMe data transfer. After successfully locking an I/O queue, the NVMe device driver synthesizes an NVMe command, enqueues the command into the associated I/O queue, and subsequently writes the tail pointer of this queue to an associated doorbell register, residing within the NVMe controller's register space, to inform the NVMe controller of the arrival of a command. Each time a doorbell rings, the controller fetches commands from the submission queue, starts processing, and writes responses to the completion queue. A legacy/MSI interrupt will also be generated if configured. After receiving an interrupt or polling the arrival of a response, the driver retrieves the response from the completion queue and writes the head pointer of the completion queue into an associated doorbell register to acknowledge completion of a data transfer. Finally, the device driver releases the allocated memory region and returned control to the OS.

2.3 Existing I/O Queuing Mechanism in NVMe Storage

With a parallel and scalable interface, NVMe is the first storage protocol that aims at significantly reducing access latencies and increasing I/O bandwidth through parallel I/O queuing to avoid synchronization overhead. In particular, while the SATA and SAS interfaces support a single queue with 32 and 256 commands, respectively, NVMe offers more with 64K queues and 64K entries per queue, thus capable of queuing and scheduling more commands and better exploiting parallel channels to access underlying storage media.

3 OVERVIEW OF DIRECTNVM SYSTEM ARCHITECTURE

In this section, we first describe potential opportunities to improve the I/O performance of conventional NVMe-based storage systems. Next, we compare the standard NVMe storage stack and our

DirectNVM architecture side-by-side to highlight various architectural innovations we designed and implemented in the DirectNVM system.

3.1 Research Motivations of DirectNVM

First, the conventional NVMe-based storage system heavily relies on OS's various services, thus adding extra performance overheads due to the multi-layered software I/O stack. As storage devices are getting faster, the legacy kernel I/O stack, originally designed for traditional HDDs, adds a noticeable overhead to the NVMe data path. As Stratikopoulos et al. [22] experimented on a Xilinx Zynq SoC, for a single NVMe I/O operation, device latency only accounts for 9%–10%, while the rest 91% of latency is distributed among I/O submission, BIO, and NVMe driver. This proves that the current I/O stack is at the same order of magnitude as device latency and is dominating I/O performance, and this trend will be even worse as more emerging storage devices being proposed, such as Intel Optane Persistent Memory [10]. This huge latency gap between the device latency and other services' latencies reveals an opportunity for NVMe I/O performance improvement. It is necessary to propose a thin I/O stack with low latency, bypassing the BIO, and with an accelerated NVMe driver functionality.

Second, NVMe scheduling logic schedules multiple request queues in a **round-robin (RR)** way. This round-robin arbitration has no control of QoS such that multiple running applications compete for NVMe bandwidth. Consequently, applications with different storage access patterns unfairly suffer different slow-downs. Even worse, a malicious application could generate many NVMe storage accesses in bursts to block other applications' I/O requests in a multi-tenant platform. Furthermore, in a real-time system where each I/O access has a latency constraint, RR is incompatible due to the lack of priority control. Another arbitration method supported by NVMe is **weighted round robin with urgent priority class (WRR)**, where queues can be classified as urgent, high, medium, or low. Different weights can be specified for each class. However, the urgency class and weight assignment is required to define at the stage of queue construction, such that they could not be dynamically changed. Traditional frameworks such as SPDK [26] propose an affinity-based priority control, in which each core is associated with a queue with a different urgency class and weight, and each application is tied to the core with the corresponding urgency class. However, the affinity-based WRR is amenable to work imbalance.

3.2 DirectNVM Architectural Innovations

Modern NVMe-based system, through optimizing queuing interface, command set, and feature set for PCIe SSDs, significantly outperforms traditional hard disks in terms of reliability, performance, and power consumption. The standard NVMe system architecture typically consists of two layers: system software and controller hardware built at the top of the PCIe interconnect. Specifically, the software layer, through the NVMe software stack/driver embedded in the OS, processes user requests and submits them to the storage device. The hardware layer, i.e., the NVMe controller, comprises various hardware modules that are responsible for dispatching requests from the NVMe driver and physically executing them on the storage device. Structurally, the software layers are executed on the cores of the main processor in the system, whereas the storage drive is connected to the host system via the PCIe Root Complex. In contrast, as shown in Figure 1(b), there are three technical innovations in the DirectNVM architecture that correspond to our observed opportunities described above, respectively.

Hardware-assisted NVMe Protocol Stack. DirectNVM designs and instantiates a custom NVMe host controller called *NVMe engine* within an FPGA device. As such, most of the standard NVMe driver's functionalities, such as command synthesis and completion polling, are realized with the FPGA hardware natively. All the components required by the standard NVMe protocol, such as I/O

queues and doorbell register ringing, are directly implemented within the NVMe hardware engine such that the whole NVMe engine is a totally self-managed independent NVMe host controller, therefore, can be used transparently on any platform even without microprocessors.

Massively Parallel Data Queues. By combining the thin kernel stack and accelerated NVMe driver functionalities, we designed a new low-latency high-throughput NVMe I/O datapath. Furthermore, by instantiating multiple independent NVMe engines, DirectNVM is compatible with multi-core architectures without conflicts on shared DRAM memory ports. More specifically, the conventional NVMe-based storage system is centrally controlled through the OS, whereas DirectNVM implements its queuing system and control scheme in a purely distributed way. Each NVMe engine inside a DirectNVM storage is completely autonomous and does not interfere with the main computing tasks of a user application running on the top of a many-core processor.

I/O-Based Weighted-Round-Robin Scheduling. By utilizing self-managed independent NVMe engines, DirectNVM designed an I/O-based WRR. For each core, up to four NVMe I/O queues are instantiated, each with a different urgency, such that the I/O traffic from each core can have all the four urgency classes. DirectNVM provides a differentiated service to each I/O to give more flexibility to users.

4 DIRECTNVM SYSTEM DESIGN

This section presents a comprehensive framework for performing userspace I/O within a DirectNVM storage system that aims at achieving high throughput and low latency. This I/O framework provides an abstract system view and an overall context for utilizing the DirectNVM architecture within a computing system. Note that, although our presented DirectNVM architecture is specially optimized for an SoC platform, it can be readily adapted to other types of computing systems, including standalone PCs or data centers.

4.1 DirectNVM I/O Framework

To achieve high bandwidth and low latency I/O through effectively utilizing the DirectNVM system architecture, we have designed a comprehensive framework for performing userspace I/O. One key design objective of our DirectNVM framework is to provide users with more flexibilities to control I/Os from each core in a multi-core computing system, such as I/O scheduling policies and I/O priorities.

The overview framework is as shown in Figure 2. Unlike FastPath [22] only supporting single-core architecture, DirectNVM was specifically designed for multi-core architecture. DirectNVM APIs, as listed in Table 2, can be easily integrated with multithreading and multiprocessing, hiding low-level details.

User-programmable Queuing System. Inspired by the multi-queue block I/O layer design in Linux kernel [3], DirectNVM contains two levels of queues with distinct functionalities. Each core has an associated staging queue to buffer generated NVMe requests temporarily. Besides, I/O scheduling, such as merging and reordering, can also be done at this stage. Currently, the framework only implements a basic FIFO I/O scheduler without request merging and reordering. However, DirectNVM supports pluggable I/O schedulers, such that more sophisticated schedulers like **completely fair queuing (CFQ)** [1] and **multi-queue deadline scheduler (mq-deadline)** can be easily integrated.

Autonomous Queuing with Parallel NVMe Engines. Inspired by NVMeDirect framework [13], to better control the I/O service, each core is associated with an I/O handle. The I/O handle controls features associated with I/O traffic from each core, such as priority, I/O buffer depth, and handles

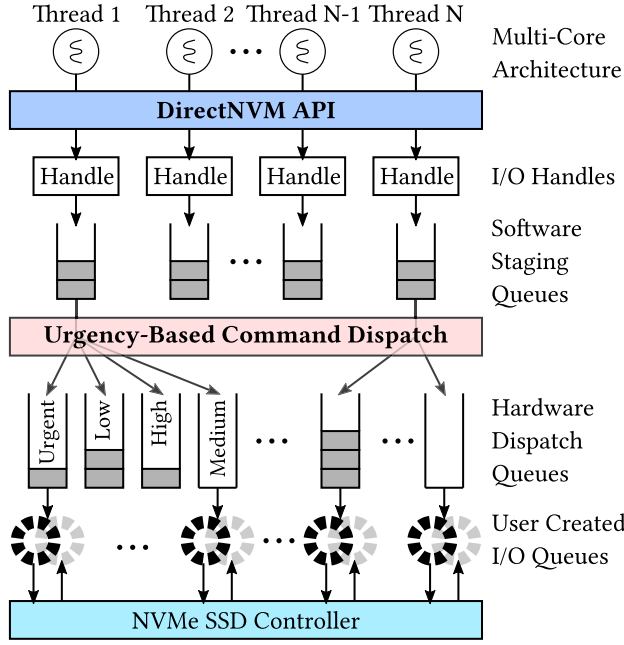


Fig. 2. Overview of DirectNVM I/O framework.

Table 2. Userspace APIs Provided by DirectNVM

API	Description
<code>nvmedl_admin_init()</code>	Initialize NVMe and pre-allocate DMA buffers and staging queues.
<code>nvmedl_create_handle(id)</code>	Create an I/O handle associated with a core.
<code>nvmedl_set_handle(handle, param, val)</code>	Set a parameter for I/O handle.
<code>nvmedl_write_async(handle, buf, lba, size)</code>	Submit an I/O write request to the staging queue.
<code>nvmedl_read_async(handle, buf, lba, size)</code>	Submit an I/O read request to the staging queue.
<code>nvmedl_poll(handle)</code>	Blocks until all outstanding requests are fulfilled.

are 1:1 mapped to software staging queues. Since interrupt-based I/O completion incurs context switching overhead, interrupt delivery delay, and additional software overhead, it is not suitable for high performance and low latency NVMe devices [15]. In DirectNVM, each I/O handle keeps polling the completion of outstanding I/O requests. The framework supports both synchronous and asynchronous I/Os.

I/O-based QoS Control. NVMe supports four urgency classes: low, medium, high, and urgent, each queue is assigned an urgency class during queue construction, and NVMe processes queues in WRR fashion. To expose the queue priority control to the userspace, each staging queue is connected with up to four hardware dispatch queues each with a different urgency class, and each NVMe request from the staging queue is dispatched to the corresponding hardware queue according to the associated urgency class. Unlike SPDK’s affinity-based WRR support method [26] that each thread needs to be affinity to the core hosting the specific type of I/O service, a thread on

DirectNVM can be scheduled on any core and thus does not suffer from load imbalance. Each hardware dispatch queue is 1:1 mapped to a submission/completion queue pair to finish I/O requests. The maximum number of hardware dispatch queues are pre-determined at the hardware design phase, and all the queues are pre-allocated on FPGA. Parameters of each buffer, such as physical address, size, are hardcoded in DirectNVM APIs. In the later release, these information might be stored in an on-chip memory on FPGA and queried by a driver code during the system bootup.

4.2 API Usage

To clearly demonstrate the flexibility of APIs provided by the DirectNVM, we now explain a simple example, as shown in Listing 1, in detailed steps. At the start of a program, the DirectNVM needs to be reset first to initialize its queues and its pre-allocated DMA data buffers. Both using `fork()` to generate multiple processes and using `pthread()` to generate multiple threads are supported inside a program. Current APIs assume that threads/processes are distributed across cores, so the number of supported threads/processes is up to the number of cores. In particular, `nvme_admin_init()` resets NVMe controller registers and NVMe acceleration engines through clearing both admin queues and I/O queues. A new I/O handle is created by passing the ID of the core on which the current process/thread is running as an argument to `nvmedl_create_handle()`. While creating this I/O handle, a pre-allocated continuous data buffer and a reserved staging queue are associated with this handle according to its CPU ID.

Some parameters of the handle are required to be configured by the user. Depth of staging queue and priority are the two important parameters. After setting handle, a user can use `nvmedl_write_async()` or `nvmedl_read_async()` to submit one I/O request to these staging queues. If the staging queue is full, then the function waits until all the current requests in the queue are all submitted and completed. Currently, DirectNVM does not support file system, so NVMe can only be accessed using **logical block address (LBA)**. The size of each logical block is configurable and manufacturer-dependent. After all requests are submitted `nvmedl_poll()` is called, and this function waits until all outstanding requests are fulfilled.

Listing 1. DirectNVM API Usage Example.

```

#include "libnvmedl.h"
#include <sched.h>

int main() {
    // initialize NVMe
    nvmedl_admin_init();
    // create handle for current core
    struct handle nvme_handle =
        nvmedl_create_handle(sched_getcpu());
    // set staging queue depth and priority
    nvmedl_set_handle(nvme_handle, "iodepth", 16);
    nvmedl_set_handle(nvme_handle, "priority", LOW);
    // read 4096 byte data from LBA 0 to data buffer
    // at address 0x800000000
    nvmedl_read_async(nvme_handle, 0x800000000, 0, 4096);
    // fulfill left requests in staging queue
    nvmedl_poll(nvme_handle);
}

```

Currently, our experimental DirectNVM system implements all the driver functionalities in the user space, such that it is not compatible with the built-in NVMe driver of Linux and only allows a

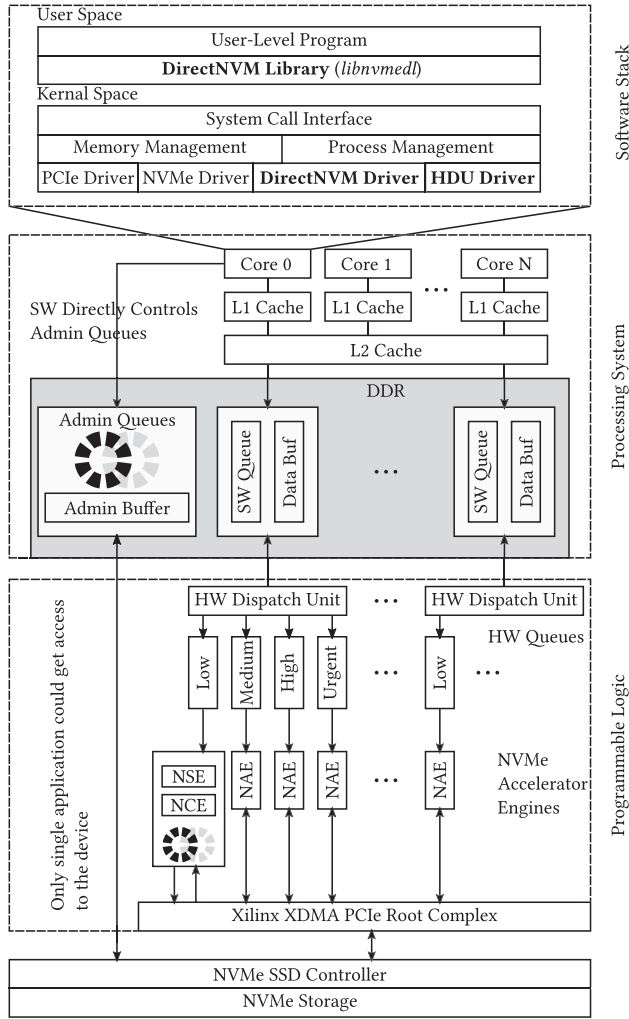


Fig. 3. Hardware architecture of DirectNVM.

single user application to get control of the NVMe device, which is similar to SPDK [26]. Moreover, the DirectNVM framework does not involve any file system, since we focus on improving raw NVMe data access performance. We leave these two tasks for our future work.

5 IMPLEMENTATION

We present more details on various key components in the hardware architecture of a DirectNVM system, as depicted in Figure 3. Directly controlled by an I/O handle, a **hardware dispatch unit (HDU)** interacts with software stack to fetch requests from software staging queues and pushes to downstream logic. An **NVMe acceleration engine (NAE)**, the central component in a DirectNVM architecture, manages NVMe I/O queues and communicates with the NVMe controller compliant with NVMe specification 1.3.

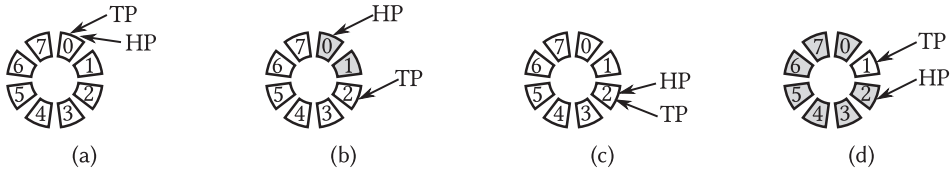


Fig. 5. NVMe queue management. HP: head pointer, TP: tail pointer. (a) Initial (empty). (b) After 2 writes. (c) After 2 reads (empty). (d) After 7 writes (full).

series of SSD benchmarking with different workloads [9] and got a conclusion that the NVMe performance asymptotically saturates as the queue depth increases. Our experiments also verify that the throughput slightly changes when the I/O queue depth grows larger than 64, which will be further explained and validated through approximate mathematical modeling in Section 7.2.

NVMe Submission Engine. **NVMe submission engine (NSE)** is responsible for constructing a submission entry for each request from the hardware dispatch queue, ringing submission tail doorbell register, and activating NCE. When an NVMe request is first streamed in from the hardware dispatch queue, a command construction unit converts each request to a 64-byte NVMe submission entry. Command construction currently only supports basic read and write operations and more NVMe commands such that flush, compare, atomic will be integrated in the future. Subsequently, each constructed NVMe submission entry is pushed into an NVMe submission circular queue, and a counter register is incremented. Meanwhile, a boundary check monitors the circular buffer to decide when to ring the submission tail doorbell register and activate the completion engine. The NVMe controller is able to fetch multiple submission entries at once in burst mode; therefore, to reduce handshaking overhead, the NSE tries to send as many submission entries to the NVMe controller as possible within a single transaction.

Each I/O circular queue has two pointers, tail and head. Tail identifies the next empty queue slot to fill, and head identifies the next queue slot to consume. For submission queue, tail is visible and controllable to user logic, while head is only visible to NVMe controller internal logic. Within the NVMe register space, there is a tail doorbell register associated with each submission queue. When one or multiple entries are pushed to the submission queue, upstream logic writes will update its tail position to its doorbell register. Subsequently, the NVMe controller compares this updated tail doorbell register with the internal head doorbell register to calculate how many new commands are submitted and fetch commands in batch from its corresponding submission queue. As shown in Figure 5, the circular queue is full when the head pointer equals one more than the tail pointer. The number of occupied entries in a queue when full is one less than the depth. So in our implementation, whenever the counter equals one less than the depth or all the commands are all sent, the submission engine stalls, signals the number of occupied entries in the current submission queue to the completion engine, and rings the tail doorbell to signal NVMe controller to start fetching commands.

NVMe Completion Engine. NVMe completion engine is mainly responsible for polling the completion of submitted commands and subsequently ringing a head doorbell. Generally, there are two ways to acknowledge the completion of each submitted command: interrupting or polling. Conventionally, the polling scheme is believed to be faster than the interrupt scheme, for the following specific reasons: (1) polling removes context switch overhead even though wasting CPU cycles to spin-wait; (2) scheduling and handling of an interrupt request involves extra overhead.

However, our study reveals that this conclusion is not directly applicable to our DirectNVM architecture because: (1) the NVMe engine is specifically designed to deal with NVMe traffic,

Table 3. Completion Entry Layout [5]

	31	23	15	7	0
DW0	Command-specific				
DW1	Reserved				
DW2	SQ Identifier		SQ Head Pointer		
DW3	Status Field	P	Command Identifier		

consequently, there is no context switching overhead. For the same reason, letting the engine spin-wait for the command completion does not hurt anything in a DirectNVM storage system; (2) unlike software interrupt handling, hardware interrupt handling has a low overhead. In contrast, our choice of polling scheme rather than interrupt is based on our study that as long as a completion is pushed to the queue, polling can immediately acknowledge while interrupt requires waiting for the delivery of the interrupt signal.

In DirectNVM, as shown in Table 3, each completion entry contains four dwords (32 bits). Among all the fields, the phase tag is a single bit at bit 16 at dword 3, which identifies if the current completion queue entry is new. All phase tags of completion entries in a completion queue are initialized to zero at the start, and whenever an NVMe controller writes a new completion message to a completion entry, its phase tag will be inverted. At all times, each NCE constantly monitors the phase tag field of the completion queue slot pointed by its head pointer. Moreover, in a DirectNVM system, each completion queue has a counter register incrementing on each new completion. When this counter equals to the number of completions to wait, which is a signal passed from the NSE, the NCE updates the head doorbell register indicating that all completions are received successfully and returns control back to NSE. Our design choice of updating a head doorbell register only once after all completions are received saves the usage of PCIe bandwidth and also reduces handshaking overhead, and this saving of PCIe bandwidth is more important when multiple queues access an NVMe device simultaneously.

6 HARDWARE PROTOTYPING

We implemented our DirectNVM framework on Xilinx ZCU106 development board equipped with Zynq UltraScale+ XCZU7EV-2FFVC1156 MPSoC, as shown in Figure 6. Four ARM A53 cores are packaged within the **processing system (PS)**. In the **processing logic (PL)**, we instantiate four hardware dispatch units and 16 NAEs, and we use submission queue creation and completion queue creation commands to signal the NVMe controller the number of required I/O queues. An M.2 NVMe SSD is attached to FPGA Drive FMC [4], and the FMC is connected to ZCU106 FPGA Mezzanine Card interface. NVMe controller uses **Xilinx DMA/Bridge Subsystem for PCI Express IP (XDMA PCIe IP)** [23] to directly access PS DDR and NAEs, and the IP is configured as a PCIe root complex. All NVMe admin queues, data buffers, and software staging queues are maintained in the DDR memory within the PS, while all HDUs, NAEs, and NVMe I/O queues are implemented on the PL. Since each NAE requires two ports to access the PCIe IP and each urgency class requires an NAE per core, to support N different urgency classes, $8N$ ports share the PCIe IP. However, each AXI IP is only allowed to connect with up to 16 connections, such that up to two urgency classes are allowed. This constraint can be resolved by using **Xilinx Integrated Block for PCI Express IP (Xilinx PCIe IP)** [25] getting rid of the AXI connection. We leave this implementation for our future release. For the OS implementation, we opted to use an embedded Linux generated and customized with the Xilinx PetaLinux SDK [24]. When customizing the embedded Linux, the device tree is customized such that all the queues and buffers are reserved

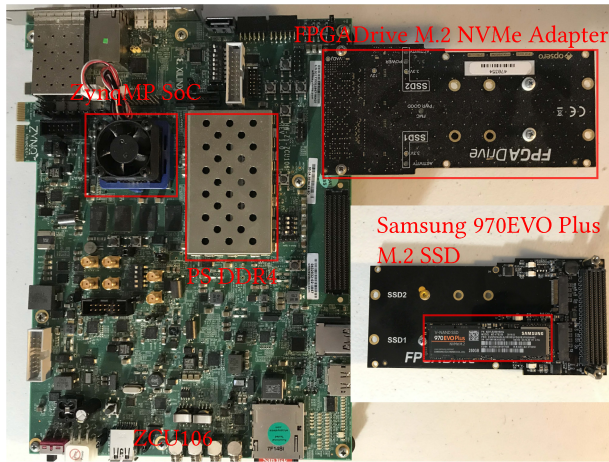


Fig. 6. DirectNVM prototype platform.

and the OS is not allowed to use these continuous memory regions under any circumstances other than under the control of associated device drivers. We developed two thread-safe Linux kernel modules, *DirectNVM driver* and *HDU driver*, and both are mounted to the embedded OS. Functionally, the *DirectNVM driver* manages reserved buffers and queues, and the *HDU driver* maps control registers of HDUs to the virtual address space at the user level. In the userspace, we developed a user-friendly C library, *libnvmmdl*, for user-level programs to use directly.

6.1 Baremetal vs. OS

Besides developing Linux drivers and APIs to integrate DirectNVM with the OS, we also designed and implemented a standalone DirectNVM library for use in a baremetal system (standalone system without an OS). Compared with a Linux-based DirectNVM system, a baremetal DirectNVM system is more lightweight and straightforward to be used in application cases where OS features are unnecessary. Instead of dealing with virtual memory address, the software stack in a baremetal DirectNVM system directly controls physical address such that it does not require memory address translation. This feature is especially useful when a PL accesses a PS memory using DMA, since the PL needs physical addresses to access a memory. However, the software stack of a baremetal DirectNVM system does not support multi-threading, memory management, and process management, which can be essential to fully exploit the performance potential of multi-core architecture, such as a Zynq UltraScale+ quad-core processor. We also evaluated baremetal implementations with different settings and compared single-core performance between the baremetal version and the OS version. The standalone library of PCIe Root Complex IP is provided by Xilinx.

6.2 Cache Coherency

An ARM processor and NAEs share data buffers and staging queues, therefore, to make sure both see the most up-to-date data, cache coherency between the processor and the NAE needs to be carefully maintained. On Zynq UltraScale+ platforms, two kinds of PS AXI slave ports, *S_AXI_HPC* and *S_AXI_HP* ports, are commonly used. The former supports hardware cache coherency while the latter does not, but the former is more complicated to implement and use. We have tested both ports and there was not much performance difference between them according to our experimental measurements, since the cache management occupies only a small portion of a total execution time.

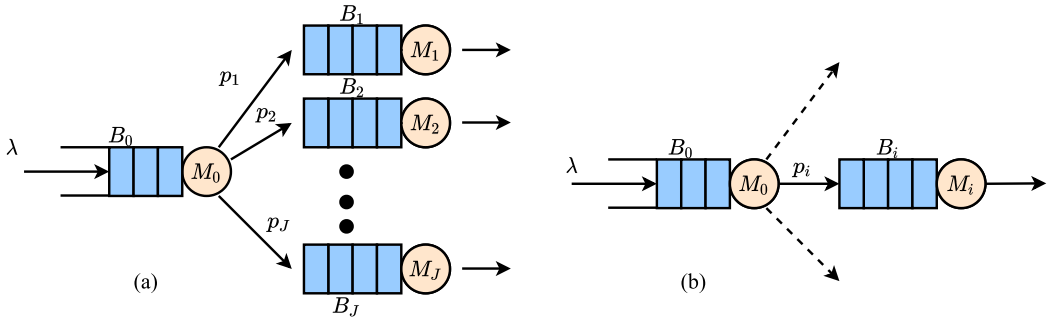


Fig. 7. (a) (b) Queuing network.

Therefore, all of our implementations are based on S_AXI_HP ports. Since HP ports do not support hardware cache coherency, in our baremetal implementation of DirectNVM, an explicit cache invalidate/flush function needs to be called before sharing data between the processor and the NAEs. In the OS version of DirectNVM implementation, a system function `pgprot_noncached()` is called to mark the memory region uncacheable, and the system function writes page attributes to the **memory management unit (MMU)**, such that all reads/writes to the marked region will not be cached. Our experimental results have shown that, compared to the manual caching management in our baremetal implementation, the OS MMU cache management provides better performance.

6.3 AXI Burst Size and Queue Depth

There are three queues used in the architecture: software staging queue, hardware dispatch queue, and NVMe I/O queue, and depths of the three queues, D_{SW} , D_{HW} , D_{IO} , are related. D_{SW} is determined by user logic, and a deeper staging queue sends more hardware requests in the batch. HDU accesses the software staging queue in AXI burst mode to reduce handshaking overhead and streams the requests into the dispatch queue. The length of each burst should be less than D_{HW} , otherwise the AXI slave will be blocked to keep waiting for sending the rest of data when the dispatch queue is full, consequently other HDUs sharing the same PS DDR AXI slave port cannot access data. In the design, we set the AXI burst length to $\frac{D_{HW}}{2}$, and whenever half of the queue slots are consumed by NAE, HDU issues an AXI burst read request to fill dispatch queue, and when dispatch queue is full, the AXI slave can serve other masters. When the NVMe submission engine starts, it attempts to read D_{IO} memory requests from the dispatch queue, constructs NVMe submission entries, and fills the submission queue. To make sure the NSE not stall waiting for the hardware queue, it requires $D_{HW} \geq D_{IO}$, and in the design, we set $D_{HW} = 2 \times D_{IO}$. D_{IO} is also determined by user logic, and in most NVMe SSD settings, it is set to 32. We implemented and evaluated four NVMe I/O queue depth settings, 8, 16, 32, 64, and we compared the performance in Section 7.

7 PERFORMANCE MODELING & RESULT ANALYSIS

7.1 Performance Modeling With Queuing Theory

Many high-dimensional parameters and complicated usage scenarios will affect the overall I/O performance of a DirectNVM storage system, therefore making its performance modeling quite challenging. To mathematically model the I/O behavior of a DirectNVM system (depicted in Figure 2), we abstract its dataflow as a two-stage open queuing network presented in Figure 7(a). Here, we only consider the data transport path between a single I/O handle and multiple hardware dispatch queues. As such, the first queuing stage is one queue with J servers and finite buffer with

buffer size B_0 , and the second queuing stage consists of J multi-server queues with finite buffer B_j , where $j = 1, \dots, J$. Clearly, the first-stage buffer and all second-stage buffers correspond to one software staging queue and multiple hardware dispatch queues, respectively (as in Figure 2); The first-stage server emulates the behavior of the urgency-based command dispatch, while the second-stage servers are functionally equivalent to the collective behavior of user-created I/O queues and the NVMe SSD controller.

Mathematically, each queuing node i consists of a server M_i and a waiting buffer of size B_i , where the index $i = 0, 1, 2, \dots, J$. To simplify mathematical derivation, we assume the service time of each server at M_i to be exponentially distributed with rate μ_i . Furthermore, all I/O command will arrive to node M_0 from an I/O handle according to a Poisson process with rate λ . For analysis simplicity, we assume that the average I/O size for each server M_i is the same. We follow the general notation and formulation as in Reference [14]. As depicted in Figure 7(b), the second stage consists of a server M_j with constant service time σ and a finite buffer of size K_j . In a DirectNVM storage system, because of **weighted round-robin (WRR)** policy and possible service blocking, the I/O commands arriving at the first server may be blocked with a blocking probability P_B . For mathematical simplicity, we model one of second stage queues as an M/D/1/K queue with arrival rate λ and constant service time σ . As such, the blocking probability P_B is given by

$$P_B = \frac{(1 - \rho)E_K}{1 - \rho E_K}, \quad (1)$$

where the load $\rho = \lambda\sigma < 1$ and under a production blocking policy

$$E_K = 1 - (1 - \rho) \sum_{j=0}^K \frac{(-1)^j \rho^j (K - j)^j e^{\rho(K-j)}}{j!}. \quad (2)$$

Finally, the throughput of this M/D/1/K queue can be calculated as $T = (1 - P_B)\lambda$. All derivation details of P_B and E_K can be found in Reference [20].

To derive the throughput of a DirectNVM storage system with WRR policy, assuming that T is the average throughput the NVMe SSD can deliver and h , m , and l are the weights of the corresponding priority classes, the average throughput for the high priority class can be calculated out as

$$T_h = \frac{(1 - \rho_h)E_{K,h}}{1 - \rho E_{K,h}}, \quad (3)$$

where $\rho_h = \sigma \cdot \frac{h}{h+m+l}$ as in Reference [7] and

$$E_{K,h} = 1 - (1 - \rho_h) \sum_{j=0}^K \frac{(-1)^j \rho_h^j (K - j)^j e^{\rho_h(K-j)}}{j!}. \quad (4)$$

7.2 Experimental Setup & Result Analysis

Our experiment is based on Samsung 970 EVO Plus NVMe SSD 250GB. Theoretical performance reported in the specification [19] is listed in Table 4. For random performance, the specification only reports **I/O per second (IOPS)** instead of bandwidth, therefore, we calculated the theoretical bandwidth by multiplying IOPS with I/O size (4 KB). Throughout the experiment, the software staging queue depth, the hardware dispatch queue depth, and the I/O queue depth are set as 64, 64, 32, by default. When measuring performance, each core separately generates I/O requests to access 32 MB data in the NVMe device, and I/O ranges of all the cores are non-overlapped. The resource utilization and clock frequency of DirectNVM with two urgency classes are shown in Table 5 and Figure 8. As mentioned in Section 6, the number of urgency classes are limited by the number of ports supported by the AXI interconnect instead of the hardware resource. We measured

Table 4. Theoretical Performance of Samsung 970 EVO Plus

Sequential Read	Sequential Write
3,500 MB/s	2,300 MB/s
Random Read (4 KB, QD32)	Random Write (4 KB, QD32)
976.56 MB/s	2,148.44 MB/s
Random Read (4 KB, QD1)	Random Write (4 KB, QD1)
66.41 MB/s	234.38 MB/s

¹QD: NVMe I/O queue depth.

Table 5. Resource Utilization of DirectNVM Supporting Two Urgency Classes (Eight NVMe Accelerator Engines in Total)

Clock Frequency Resource Utilization (Utilization%)			
250 MHz	LUT	FF	BRAM
	59,348 (26%)	68,112 (15%)	186.50 (60%)

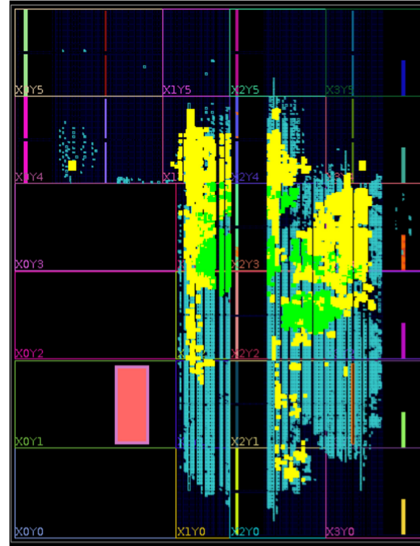


Fig. 8. Resource utilization of DirectNVM with two urgency classes. Yellow highlighted are eight NVMe accelerator engines (NAEs), green highlighted are four hardware dispatch units (HDUs), and red highlighted is the ZynqMP processor.

the throughput and average latency for both **baremetal implementation (DL-Baremetal)** and **OS implementation (DL-OS)**. We used PetaLinux I/O stack and the built-in NVMe driver as the baseline. We measured the performance of the baseline using **Flexible I/O (FIO)** benchmark, a common tool to test Linux I/O subsystem and schedulers [2].

Single-core Performance. We measured the single-core throughput of both baremetal implementation and OS implementation, and compared with the baseline, as shown in Figure 9(a). DirectNVM OS version and baremetal version both greatly outperform the baseline. We also compared

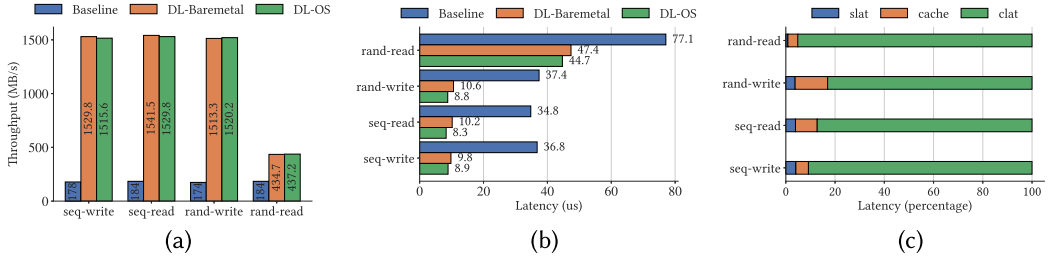


Fig. 9. (a) Single-core throughput comparison. (b) Single-core average latency comparison. (c) Single-core average latency breakdown.

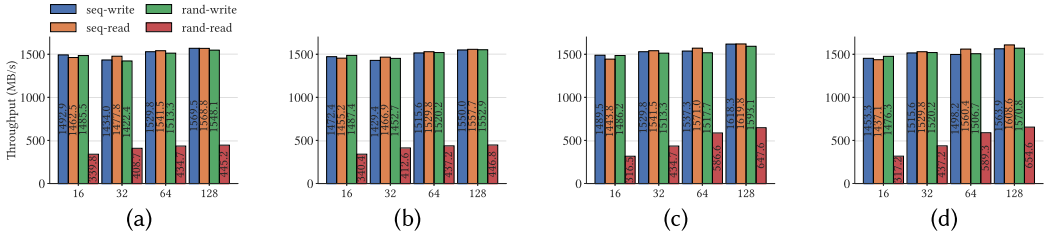


Fig. 10. (a) (b): Impact of software staging queue depth on baremetal/OS single-core throughput. (c) (d): Impact of NVMe I/O queue depth on baremetal/OS single-core throughput.

the average latency. During the comparison, we set the software staging queue depth to 1 such that each NVMe request is processed in a blocking way. As shown in Figure 9(b), DL-Baremetal and DL-OS achieve up to 3× speedup over the baseline. DL-OS has a slightly shorter latency than DL-Baremetal because of the efficient cache management provided by MMU.

We further did a latency breakdown analysis for the baremetal implementation, as shown in Figure 9(c). For a single request, we measured three latencies: *slat*, *cache*, and *clat*. *slat* measures the latency to submit a single request to the software staging queue, *cache* measures the latency to invalidate and flush the software staging queue to maintain consistency, *clat* measures the latency that the DirectNVM subsystem processes the submitted request. As shown in Figure 9(c), *clat* dominates the average single-request latency, and the cache invalidate/flush latency is not the bottleneck. This also explains the ignorable latency difference between DL-OS and DL-Baremetal. We further studied the impacts of software staging queue depth and NVMe I/O queue depth on the system performance for both baremetal implementation and OS implementation. As shown in Figures 10(c) and 10(d), when increasing NVMe I/O queue depth, the throughput is increased and when the NVMe I/O queue is deeper than 64, the throughput increases slowly. This proves the feasibility of our approach implementing NVMe I/O queues using on-chip block RAM, since a deeper I/O queue will only improve the throughput by 4%. As shown in Figures 10(a) and 10(b), the single-core throughput slightly increases when increasing the depth of the software staging queue. When the software staging queue is deeper than 64, the performance increases very slowly.

Multi-core Performance. Next, we compared the multi-core system performance with the baseline and also compared with the theoretical performance reported in the SSD specification. As shown in Figure 11(a), DirectNVM greatly outperforms the baseline when writing to the device, and the write throughput achieves 80% of the theoretical bandwidth. When randomly reading from SSD, the random performance is slightly better than the baseline and it achieves 60% of the

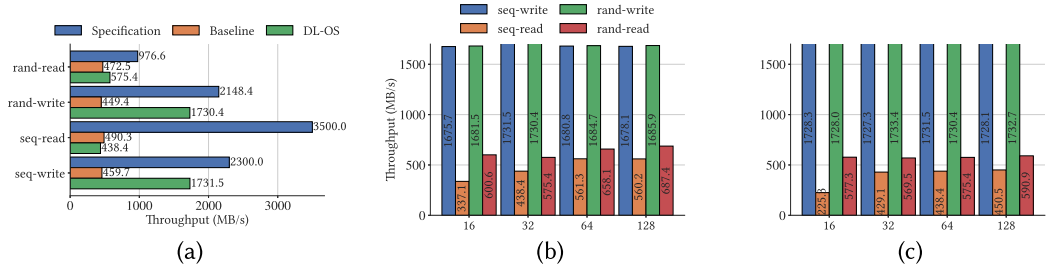


Fig. 11. (a) Quad-core throughput comparison. (b) Impact of NVMe I/O queue depth on quad-core throughput. (c) Impact of software staging queue depth on quad-core throughput.

theoretical bandwidth, and sequential performance is almost the same as the baseline implementation. The performance gap between the sequential read and the random read are mainly affected by the SSD built-in queue scheduler, and we also found the same comparison result in another literature using the same generation device [16]. When there are multiple pending queues, the scheduling among the queues is determined by the SSD queue scheduling logic, and we give a reasonable explanation that this multiple sequential access pattern cannot exploit the SSD internal parallelism such that the throughput is even lower than the random read, and we expect this result is vendor-dependent.

The impacts of the NVMe I/O queue depth and the software staging queue depth on the quad-core performance are shown in Figures 11(b) and 11(c). Same as the single-core performance, the system throughput increases very slowly when the NVMe I/O queue is deeper than 64, and the throughput keeps almost constant when the software staging queue grows deeper than 64. In Figure 11(b), the performance is slightly better when the queue depth is 32, and we consider this a normal variance across multiple measurements. In the experiment, we found one observation that was not mentioned in other literature. We found that the assigned urgency class could heavily affect system throughput. When assigning multiple queues with the same urgency class, multiple queues are processed by the NVMe SSD device in a round-robin way. However, we found that even scheduled in the round-robin fashion, different urgency classes affected the system throughput. We did two experiments, assigning “urgent” and “high” urgency classes to all the queues, and we measured the performance. As shown in Figure 12, when assigning “urgency” to the queues, the random write and sequential write throughput are higher than assigning “high” to the queues. The random write performance even outperforms the theoretical bandwidth reported in the specification. However, the sequential read and random read performances are worse when assigning “urgency.” We are suspecting this performance gap is caused by the vendor-specific SSD scheduler, and we will investigate the reason for this performance gap in our following work.

Differentiated I/O Service. One benefit of DirectNVM is to expose the NVMe weighted round-robin queue scheduling control to the userspace to provide differentiated I/O service and **quality of service (QoS)** control. This per-core I/O scheduling provides flexibility to control the urgency of each core’s traffic. In this experiment, we use two threads to represent a foreground application and a background application, and the background thread issues two burst NVMe requests. This simulated experiment setting happens in a lot of daily NVMe usage, e.g., a foreground application is processing data while a background network application is downloading media from the network. In most of this kind of use case, the foreground application’s throughput is preferred to maintain. In the first experiment, we assigned the two threads “high,” and as shown in Figure 13(a), when the background thread is active, the foreground throughput is heavily decreased.

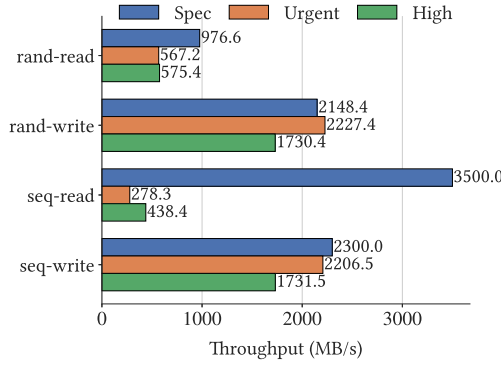


Fig. 12. Impact of urgency class on system throughput.

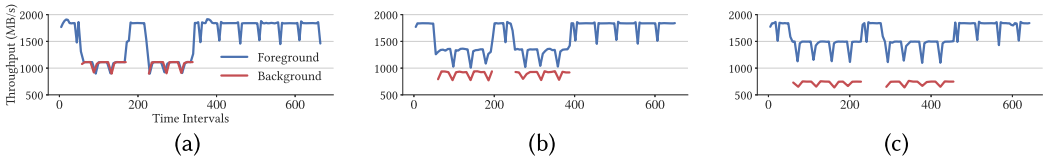


Fig. 13. Differentiated service of two threads (foreground and background) provided by DirectNVM. (a) shows two threads without weighted round robin control. (b) and (c) are weighted round robin control by using DirectNVM APIs by setting the weight as 8:1 and 128:1, respectively.

Then, in the userspace code, we assigned the foreground thread as “high” and the background thread as “medium” when it starts. As shown in Figures 13(b) and 13(c), the foreground thread throughput is less affected, since the weighted round-robin scheduling allocates more bandwidth to the “high” urgency class. We studied the impact of weights assigned to each urgency class. The weight of “medium” urgency class was assigned to 1, and we adjusted the “high” urgency class. As shown in Figures 13(b) and 13(c), when assigning the weight of “high” urgency class as 128, the foreground thread is higher than setting the weight to 8. DirectNVM provides a flexible way for the user to directly control each core’s differentiated I/O service.

FIO Integration and Comparison against SPDK. To further prove the performance improvement of DirectNVM, we did a comparison with Intel SPDK [26]. Intel SPDK is a widely utilized state-of-the-art userspace I/O framework that achieves optimized NVMe I/O performance through bypassing the OS kernel stack. To benchmark with the FIO package, we implemented an FIO plugin for the DirectNVM framework using the DirectNVM API and the FIO API, while the SPDK FIO plugin is officially provided by Intel. In our experiment, the depth of staging queue, hardware dispatch queue, and NVMe I/O queue are set as 64, 128, and 64, respectively. In our experiments, we ran SPDK on a desktop PC equipped with an AMD Ryzen 2700x CPU and a Samsung 970 EVO Plus SSD. Unlike our DirectNVM, the Intel SPDK FIO plugin does not support multi-threading on single NVMe namespace, thus, we only compared single-thread performance. To reduce the effect of internal caching as well as to avoid repeatedly accessing a cached data block, we first tested the sequential write and the random write and then tested the sequential read and the random read. The I/O size was set to 4 KB, and we used FIO to access a 10 GB data range. As shown in Figures 14(a) and Figure 14(b), we measured both the single-thread throughput and latency of DirectNVM and SPDK. Due to the significant software overhead, when benchmarking with the FIO package, we

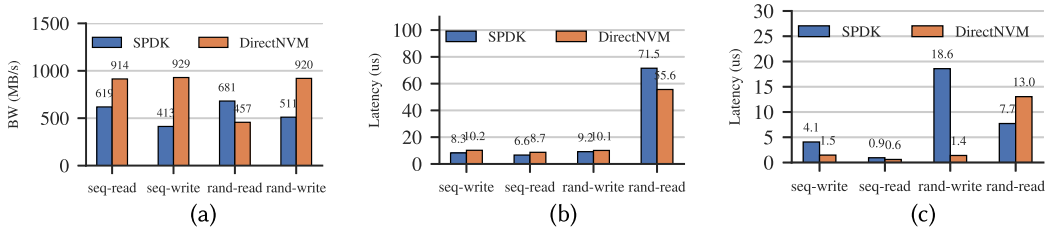


Fig. 14. (a) Single-thread throughput comparison. (b) Single-thread average latency comparison. (c) Standard deviation of single-thread latency.

found that the I/O bandwidth of both DirectNVM and SPDK were decreased. Maybe most importantly, we found that, compared to the SPDK running on a full-scale processor equipped with the same NVMe SSD, our DirectNVM system outperforms for most test cases. However, our DirectNVM's random read bandwidth surprisingly dropped possibly due to two reasons: First, unlike traditional hard drives and old-generation SSDs, when using the newest-generation SSDs, 4K random read normally has lower bandwidth than 4K random write. This is likely due to the fact that the newer-generation NVMe SSD has an internal DRAM cache to temporarily buffer written data, thus reducing the impact of garbage collection. In contrast, when randomly reading 4K blocks from a SSD device, this internal cache cannot predict the next block to prefetch such that random read has a lower bandwidth. This hypothesis can be partially corroborated from the specification shown in Table 4. Note that, in this specification, no matter how the NVMe I/O queue depth is set, 4K random read has a lower maximum bandwidth than 4K random write. Similarly, Miemietz et al. [16] observed the same experiment result of the low 4K random read bandwidth using a same-generation Samsung SSD as ours and concluded with the same hypothesis. Second, SPDK adopts an asynchronous framework implemented with two separate threads, with one controlling NVMe request synthesis and submission while the other processing NVMe completion polling. As such, SPDK can effectively overlap I/O command submission and completion. In contrast, our proposed DirectNVM system adopts a batched synchronous framework where all the NVMe requests are temporarily buffered in a software staging queue and then pushed to the hardware NVMe controller in batches. Consequently, only when all the outstanding requests are completed, DirectNVM will generate and submit new I/O requests.

To experimentally compare the average I/O latency between the two systems, we have set the software staging queue depth to 1 for all the systems such that each NVMe request is processed in a blocking way. As such, SPDK's FIO configuration was also changed accordingly. As shown in Figure 14(b), SPDK running on the PC has a slightly shorter latency than DirectNVM on Zynq MPSoC, since SPDK has less software overhead due to its asynchronous completion polling and higher CPU clock frequency. In addition, in DirectNVM, all the commands are sent from host to the controller over AXI interconnect, adding a lot of overhead to the processing of each request. Since each single I/O request is processed in a blocking way when testing the latency, the overhead of AXI traffic is exacerbated compared with submitting requests in batches. However, due to our DirectNVM's highly efficient NVMe controller design and system framework, the performance gap between our DirectNVM and SPDK is almost negligible. Note that our DirectNVM is running on a clock frequency of 250 MHz, thus the latency gap is only up to 13 clock cycles. As shown in Figure 14(c), even though our DirectNVM has a slightly longer latency than SPDK, our DirectNVM's latency has a much lower variance than SPDK, since most of NVMe driver functionalities are moved to hardware not relying on OS scheduling, which can be critical to some real-time and latency-constraint applications. However, when comparing the latency of a 4K random read, our

DirectNVM has a 5.3 μ s higher variance than SPDK. Taking our FPGA clock frequency 250 MHz into consideration, the 5.3 μ s variance only indicates 1.3 clock cycles, which is negligible to the hardware implementation. We surmise that, by a more dedicated optimization, DirectNVM can achieve further lower latency than SPDK.

8 FUTURE WORK FOR POTENTIAL EXTENSIONS

NVMe I/O Queue Mapping—The standard NVMe protocol facilitates I/O scheduling with five different urgency groups. Our current DirectNVM allocates a separate queue for each urgency per core, thus the scalability is limited by the number of I/O queues that the PL can accommodate. However, we have find that multiple priority groups can reuse the same NVMe I/O queue because only head and tail pointers are different across different priorities. By sharing the same I/O queue and NVMe controller among different priorities for each processor core, the ratio between the number of processing cores and the number of I/O queues can be decreased from 1:4 to 1:1, thus significantly saving hardware resource. We leave this optimization to our future work.

QoS Control— Since the current implementation of our DirectNVM system submits I/O requests in a synchronous and batch mode, all these requests within a batch have to be of the same priority, while different batches are allowed to have different priorities. This operational limitation can be addressed by partitioning a batch of commands with different priorities into multiple sub-batches when using our DirectNVM API. As such, we plan to integrate a finer-grained QoS control into the future release of DirectNVM.

9 CONCLUSIONS

In this article, we have introduced the DirectNVM system, a hardware-accelerated NVMe-based architecture of SSD storage that effectively achieves low-latency and high-bandwidth data communication for high-performance embedded computing system. Our key idea is to exploit the massively parallel I/O queuing capability, fine-grained control, high processing concurrency, and application-specific optimization within a modern NVMe storage system through leveraging FPGAs' reconfigurability and native hardware computing power. We have designed and implemented a prototype DirectNVM storage system with the mainstream Xilinx MPSoC technology that incorporates high-performance **application processors (APUs)** with DDR4 system memory and PCIe Gen3 blocks. Our prototype demonstrates that the DirectNVM storage system reduces latency by up to 4.5 $\times$ compared to a standard baseline. Furthermore, the DirectNVM storage architecture can help significantly reduce the cost and power overhead of large-scale storage nodes in data centers. The demo and source code of our DirectNVM system are available at Reference [27].

REFERENCES

- [1] Jens Axboe. 2006. Linux Kernel 2.6.18 - Make CFQ the default IO scheduler. Retrieved from <https://git.kernel.org/cgit/linux/kernel/git/torvalds/linux.git/commit/?id=b17fd9bceb99610f6dc7998c9a4ed6b71520be2b>.
- [2] Jens Axboe. 2020. Flexible I/O. Retrieved from <https://github.com/axboe/fio>.
- [3] Matias Björling, Jens Axboe, David Nellans, and Philippe Bonnet. 2013. Linux block IO: introducing multi-queue SSD access on multi-core systems. In *Proceedings of the 6th International Systems and Storage Conference*. 1–10.
- [4] Opsero Electronic Design. 2020. FPGA Drive FMC. Retrieved from <https://opsero.com/product/fpga-drive-fmc-dual/>.
- [5] NVMe Express. 2017. NVMe Specification 1.3. Retrieved from https://nvmexpress.org/wp-content/uploads/NVMe_Express_Revision_1.3.pdf.
- [6] OpenPOWER Accelerator Work Group. 2020. CAPI Storage, Network, and Analytics Programming (SNAP) Framework. Retrieved from <https://developer.ibm.com/linuxonpower/capi/snap>.
- [7] Shashank Gugrani, Xiaoyi Lu, and Dhabaleswar K. D. K. Panda. 2019. Analyzing, modeling, and provisioning QoS for NVMe SSDs. In *Proceedings of the 11th IEEE/ACM International Conference on Utility and Cloud Computing*. 247–256. DOI: <https://doi.org/10.1109/UCC.2018.00033>

- [8] Jeremy Hsu. 2018. It's Time to Think Beyond Cloud Computing. Retrieved from <https://www.wired.com/story/its-time-to-think-beyond-cloud-computing/>.
- [9] Intel. 2015. Performance Benchmarking for PCIe and NVMe Enterprise Solid-State Drives. Retrieved from <https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/performance-pcie-nvme-enterprise-ssds-white-paper.pdf>.
- [10] Intel. 2020. Intel Optane Persistent Memory. Retrieved from <https://www.intel.com/content/www/us/en/architecture-and-technology/optane-dc-persistent-memory.html>.
- [11] Intel. 2020. Open Programmable Accelerator Engine. Retrieved from <https://opae.github.io/latest/index.html>.
- [12] Yangwook Kang, Yang-suk Kee, Ethan L. Miller, and Chanik Park. 2013. Enabling cost-effective data processing with smart SSD. In *Proceedings of the IEEE 29th Symposium on Mass Storage Systems and Technologies (MSST)*. IEEE, 1–12.
- [13] Hyeon-Jun Kim, Young-Sik Lee, and Jin-Soo Kim. 2016. NVMeDirect: A user-space I/O framework for application-specific optimization on nvme SSDs. In *Proceedings of the 8th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage'16)*.
- [14] László Lakatos, László Szeidl, and Miklós Telek. 2013. *Introduction to Queueing Systems with Telecommunication Applications*. Vol. 9781461453. Springer. DOI: <https://doi.org/10.1007/978-1-4614-5317-8>
- [15] Damien Le Moal. 2017. I/O Latency Optimization with polling. In *Proceedings of the Vault Linux Storage and Filesystems Conference*.
- [16] Till Miemietz, Hannes Weisbach, Michael Roitzsch, and Hermann Härtig. 2019. K2: Work-constraining scheduling of NVMe-attached storage. In *Proceedings of the IEEE Real-time Systems Symposium (RTSS)*. IEEE, 56–68.
- [17] Arslan Munir, Sanjay Ranka, and Ann Gordon-Ross. 2012. High-performance energy-efficient multicore embedded computing. *Trans. Parallel Distrib. Syst.* 23 (05 2012), 684–700. DOI: <https://doi.org/10.1109/TPDS.2011.214>
- [18] Zhenyuan Ruan, Tong He, and Jason Cong. 2019. INSIDER: Designing in-storage computing system for emerging high-performance drive. In *Proceedings of the USENIX Annual Technical Conference (USENIXATC'19)*. 379–394.
- [19] Samsung. 2020. Samsung 970 EVO Plus Specification. Retrieved from <https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/970evoplus/>.
- [20] Dong Won Seo. 2014. Explicit formulae for characteristics of finite-Capacity M/D/1 queues. *ETRI J.* 36, 4 (2014), 609–616. DOI: <https://doi.org/10.4218/etrij.14.0113.0812>
- [21] Athanasios Stratikopoulos. 2019. *Low Overhead & Energy Efficient Storage Path for Next Generation Computer Systems*. Ph.D. Dissertation. University of Manchester.
- [22] Athanasios Stratikopoulos, Christos Kotselidis, John Goodacre, and Mikel Luján. 2018. FastPath: Towards wire-speed NVMe SSDs. In *Proceedings of the 28th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 170–1707.
- [23] Xilinx. 2019. DMA/Bridge Subsystem for PCI Express v4.1. Retrieved from https://www.xilinx.com/support/documentation/ip_documentation/xdma/v4_1/pg195-pcie-dma.pdf.
- [24] Xilinx. 2019. PetaLinux Tools Documentation. Retrieved from https://www.xilinx.com/support/documentation/sw_manuals/xilinx2019_2/ug1144-petalinux-tools-reference-guide.pdf.
- [25] Xilinx. 2019. UltraScale+ Devices Integrated Block for PCI Express v1.3. Retrieved from https://www.xilinx.com/support/documentation/ip_documentation/pcie4_uscale_plus/v1_3/pg213-pcie4-ultrascale-plus.pdf.
- [26] Ziye Yang, James R. Harris, Benjamin Walker, Daniel Verkamp, Changpeng Liu, Cunyin Chang, Gang Cao, Jonathan Stern, Vishal Verma, and Luse E. Paul. 2017. SPDK: A development kit to build high performance storage applications. In *Proceedings of the IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, 154–161.
- [27] Yu Zou. 2020. DirectNVM. Retrieved from <https://github.com/yu-zou/DirectNVM>.

Received December 2020; revised March 2021; accepted April 2021